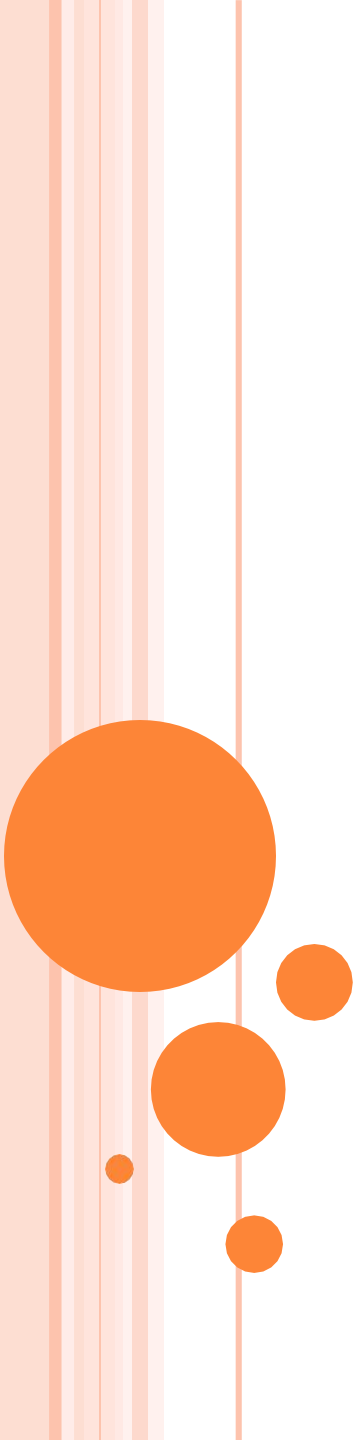




UNIT-2

BUILDING BLOCKS OF THE LANGUAGE

TOPICS TO BE COVERED...

1. Primitive Data Types : Integers, Floating Point type, Characters, Booleans etc
2. User Defined Data Type
3. Identifiers & Literals
4. Declarations of constants & variables
5. Type Conversion and Casting
6. Scope of variables & default values of variables declared
7. Wrapper classes
8. Comment Syntax
9. Garbage Collection

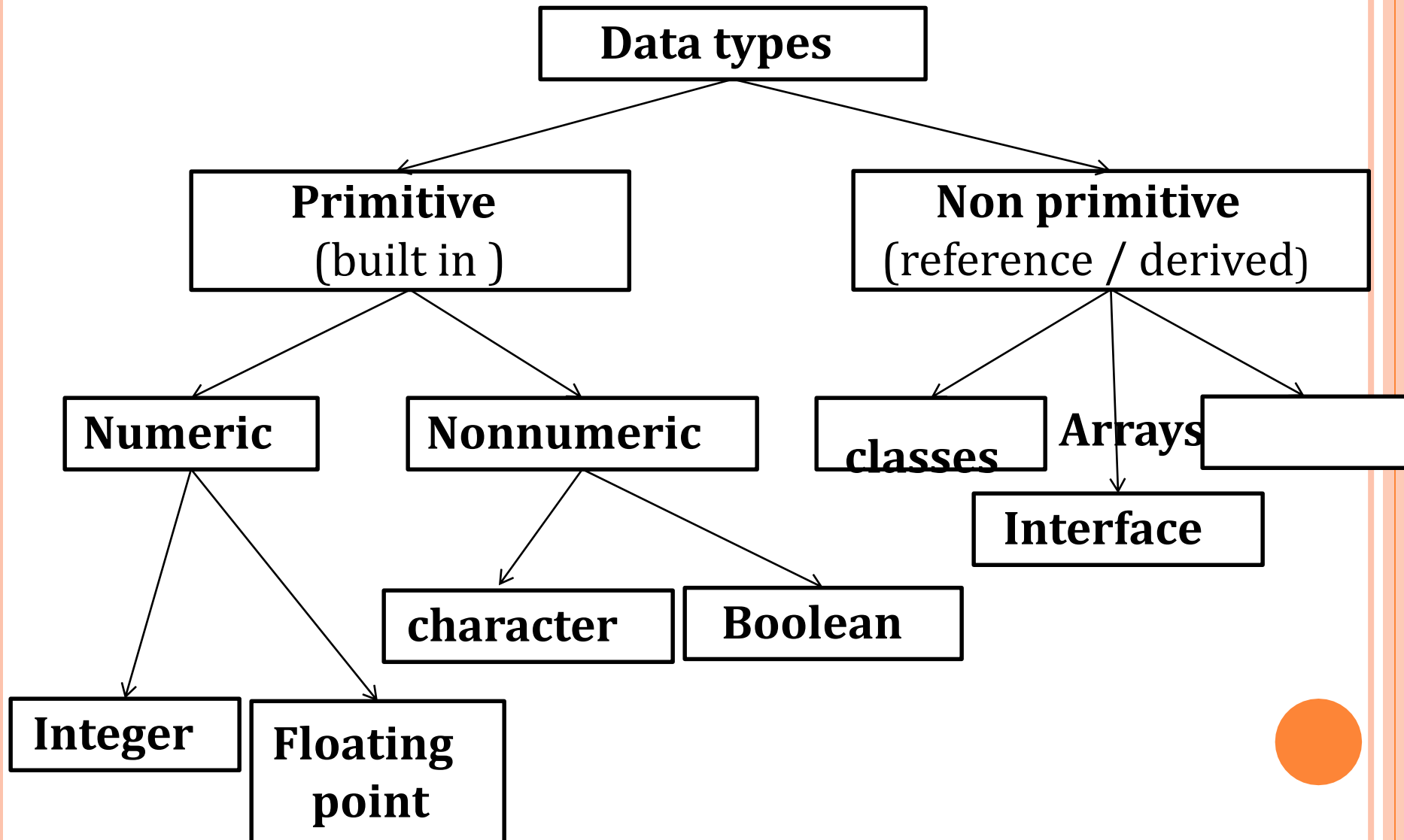


TOPICS TO BE COVERED...

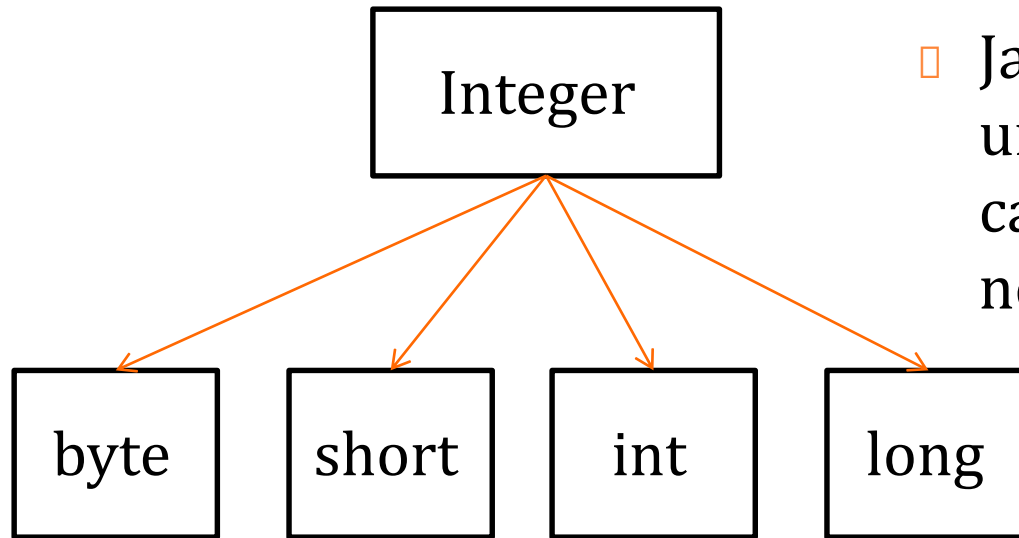
10. Arrays of Primitive Data Types
11. Types of Arrays
12. Creation, concatenation and conversion of a string, changing case of string, character extraction, String Comparison, String Buffer
13. Different Operators: Arithmetic, bitwise, Relational, Logical, Assignment, Conditional, ternary, Increment/Decrement, Mathematical Functions
14. Decision and control statements: Selection Statement (if, if...else, switch), Loops while, do- while, for), Jump statements (break, continue, return & exit)



2.1 DATA TYPES IN JAVA



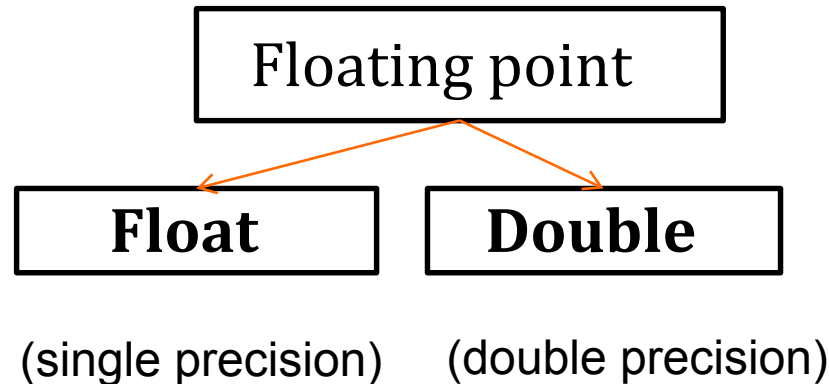
INTEGER TYPES



- Java does not support unsigned type, so values can be positive or negative.

Type	Size	Minimum value	Maximum value
byte	1 byte	-128	127
short	2 byte	-32768	32767
int	4 byte	-2,147,483,648	2,147,483,647
long	8 byte	-9,223,372,036,854,754,808	9,223,372,036,854,754,807

FLOATING TYPES



Type	Size	Minimum value	Maximum value
Float	4 byte	3.4 e -038	3.4 e +038
double	8 byte	1.7 e -308	1.7 e +308



- Floating point no. are treated as **double precision** , so to force them to be in single precision mode ,we must append **f or F**.
- Ex: 1.23f , 7.6512F
- **NaN: Not a Number**

(special value supported by floating point data type)

It is used to represent result of operation such as dividing by zero ,where actual number is not produced.



CHARACTER TYPE

- ❑ **char** is used
- ❑ Size : 2 byte
- ❑ It holds single character only.

BOOLEAN TYPE

- ❑ It is used to test a particular condition during execution of program.
- ❑ It has two values : true or false
- ❑ boolean keyword used
- ❑ Size: 1 bit (0 or 1)
- ❑ All comparison operators return boolean type variable.



- So, there are **8 basic(primitive)** data type in java.

Sr. No.	Type	Size (in bytes)
1	boolean	1 bit
2	byte	1 byte
3	character	2 bytes
4	short	2 bytes
5	int	4 bytes
6	float	4 bytes
7	long	8 bytes
8	double	8 bytes



2.2 USER DEFINED DATA TYPE

- ❑ **Class, interface and array are user defined data type.**
- ❑ Class encapsulates various variables and methods
- ❑ Interface is same as class except it contains methods without body. It is used to implement multiple inheritance.
- ❑ An array contains same type of elements and provides sequential access.



Java tokens:

- Java program is a collection of classes.
- Class is defined by a set of declaration statements and methods containing executable statements.
- Smallest individual unit in program is tokens.
- Compiler recognize them for building up expressions and statement
- *Java program=tokens + comments + white space*



2.3 IDENTIFIERS & LITERALS

- The only allowed characters for identifiers are all alphanumeric characters([**A-Z**],[**a-z**],[**0-9**]), '**\$**'(dollar sign) and '**_**' (underscore). For example “geek@” is not a valid Java identifier as it contains a '@' a special character.
- Identifiers should **not** start with digits([**0-9**]). For example “123ab” is not a valid Java identifier.
- Java identifiers are **case-sensitive**.
- There is no limit on the length of the identifier but it is advisable to use an optimum length of 4 – 15 letters only.
- Reserved Words** can't be used as an identifier. For example “int while = 20;” is an invalid statement as a while is a reserved word. There are **53** reserved words in Java



Example

```
public class Test
{
    public static void main(String[] args)
    {
        int a = 20;
    }
}
```

In the above Java code, we have 5 identifiers as follows:

- **Test**: Class Name
- **main**: Method Name
- **String**: Predefined Class Name
- **args**: Variable Name
- **a**: Variable Name



Literals in Java

Decimal literals (Base 10): In this form, the allowed digits are 0-9.

```
int x = 101;
```

Octal literals (Base 8): In this form, the allowed digits are 0-7.

```
// The octal number should be prefix with 0. int x = 0146;
```

Hexa-decimal literals (Base 16):

```
// The hexa-decimal number should be prefix // with 0X or 0x. int x = 0X123Face;
```

Binary literals

```
int x = 0b1111;
```



Floating-Point literal

Float b=5.2f;

```
double d = 123.456;
```

Char literals

```
char ch = 'a';
```

```
char ch = '\u0061';// Here /u0061 represent a.
```

```
char ch = '\n';
```

String literals

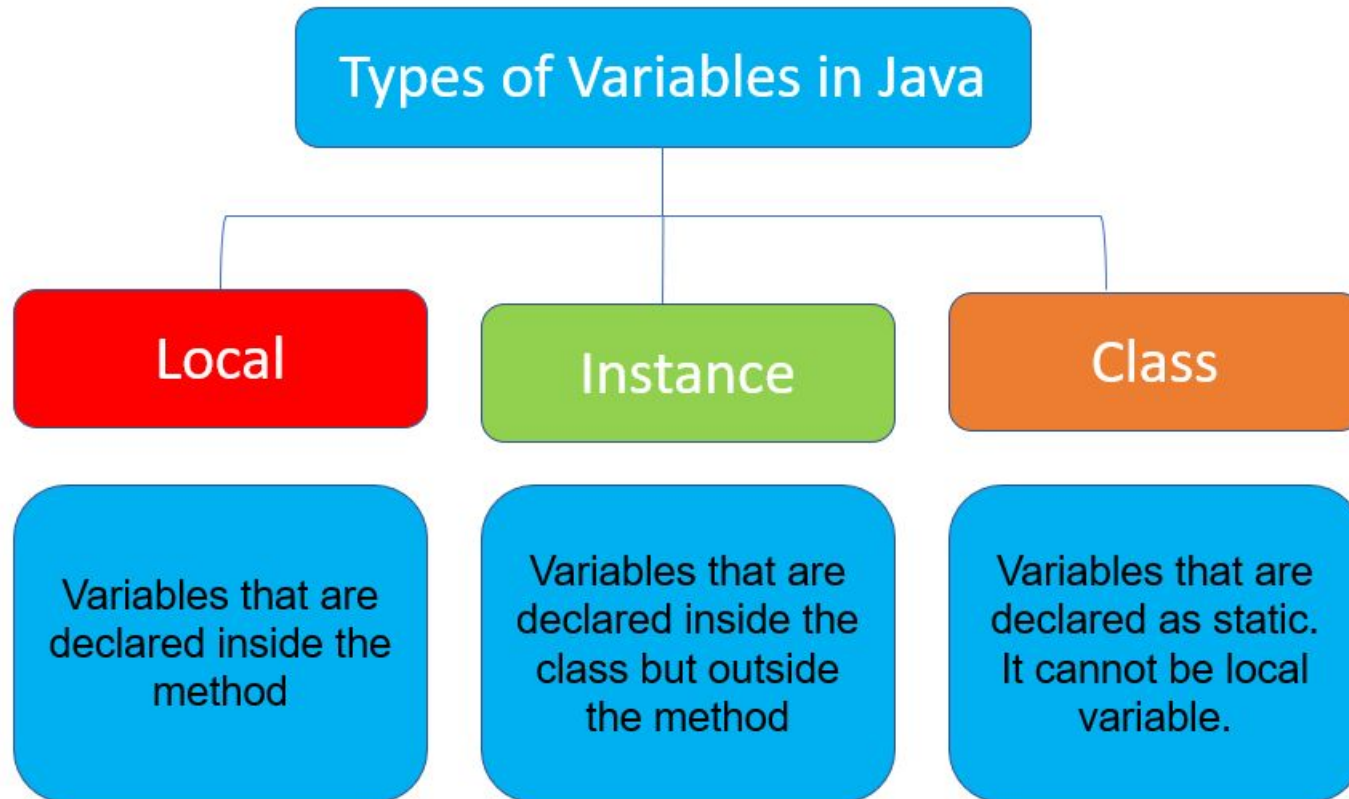
```
String s = "Hello";
```

Boolean literals

```
boolean b = true;
```



Java Variables



How to Declare Java Variables?

```
int a=150;
```

```
Int a;
```

```
A=50;
```

Rules for naming variable

How to Initialize Java Variables?

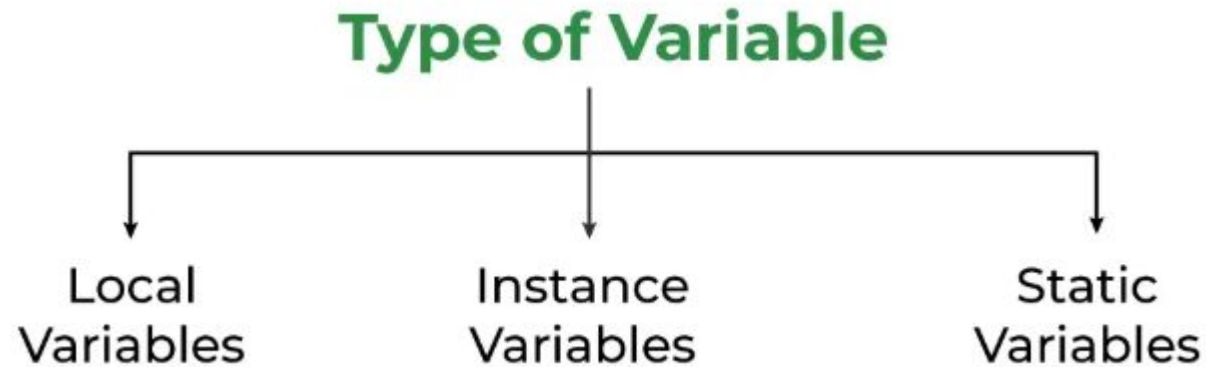
```
// Declaring float variable  
float simpleInterest;
```

```
// Declaring and initializing integer variable  
int time = 10, speed = 20;
```

```
// Declaring and initializing character variable  
char var = 'h';
```



SCOPE OF VARIABLE & DEFAULT VALUES



Types of Java Variables



1) Local Variable

A variable declared inside the body of the method is called local variable.

You can use this variable only within that method and the other methods in the class aren't even aware that the variable exists.

A local variable cannot be defined with "static" keyword.

```
public class LocalVariableExample
{
    public static void main(String[] args)
    {
        //defining a Local Variable
        int num = 10;
        System.out.println(" Variable: " + num);
    }
}
```



2) Instance Variable

Instance variables are non-static variables and are declared in a class outside of any method, constructor, or block.

Initialization of an instance variable is not mandatory. Its default value is dependent on the data type of variable. For *String* it is *null*, for *float* it is *0.0f*, for *int* it is *0*

Unlike local variables, we may use access specifiers for instance variables. If we do not specify any access specifier, then the default access specifier will be used

Scope of instance variables are throughout the class except the static contexts.



```
import java.io.*;

class GFG {

    // Declared Instance Variable
    public String a;
    public int i;
    public Integer l;
    public GFG()
    {
        // Default Constructor
        // initializing Instance Variable
        this.a = "Shubham Jain";
    }

    // Main Method
    public static void main(String[] args)
    {
        // Object Creation
        GFG name = new GFG();

        // Displaying O/P
        System.out.println("Geek name is: " + name.a);
        System.out.println("Default value for int is "+ name.i);

        // toString() called internally
        System.out.println("Default value for Integer is "+ name.l);
    }
}
```



3. Static Variables

- These variables are declared similarly to instance variables. The difference is that static variables are declared using the `static` keyword within a class outside of any method, constructor, or block.
- Static variables are created at the start of program execution and destroyed automatically when execution ends.
- Initialization of a static variable is not mandatory. Its default value is dependent on the data type of variable. For *String* it is *null*, for *float* it is *0.0f*, for *int* it is *0*, for *Wrapper classes* like *Integer* it is *null*, etc.



```
import java.io.*;

class GFG {
    // Declared static variable
    public static String a = "Vaidehi Shah";

    public static void main(String[] args)
    {

        // a variable can be accessed without object
        System.out.println(" Name is : " + GFG.a);

    }
}
```



Java Tokens

- 1) reserved keywords
- 2) identifiers
- 3) literals
- 4) operators
- 5) separators



KEYWORDS

- Java has 60 reserved keywords.
- We can not use keywords as names for variables, classes, methods and so on.
- Keywords should be written in small letters.

abstract	Boolean	break	byte	byvalue*	case	cast*
catch	char	class	const*	continue	default	do
double	else	extends	false**	final	finally	float
for	future**	generic*	goto*	if	implements	import
inner*	instanceof*	int	interface	long	native	new
null**	operator*	outer*	package	private	protected	public
rest*	return	short	static	super	switch	try
this	threadsafe*	throw	throws	transient	true**	
var*	void	volatile	while		synchronized	

* Reserved for future use

** These are values defined by java



IDENTIFIERS

- Identifiers are programmer designed tokens.
- It is used for naming classes, methods , variables , objects , label , package and interface in program.
- Example:
 - average , sum , dayTemperature , totalMarks

LITERALS

- Sequence of characters (digits , letters ,and other character) that represent constant values to be stored in variables.
 - Five types:
 - Integer literals
 - Floating point literals
 - Character literals
 - String literals
 - Boolean literals
- 

Operators

- All operator is a symbol that takes one or more arguments and operates on them to produce a result.

Seperators

- They are symbols used to indicate where groups of code are divided and arranged.

Sr. No.	Name
1	Parentheses ()
2	Braces { }
3	Brackets []
4	Semicolon ;
5	Comma ,
6	Period .



2.4 DECLARATIONS OF CONSTANTS & VARIABLES

Constant

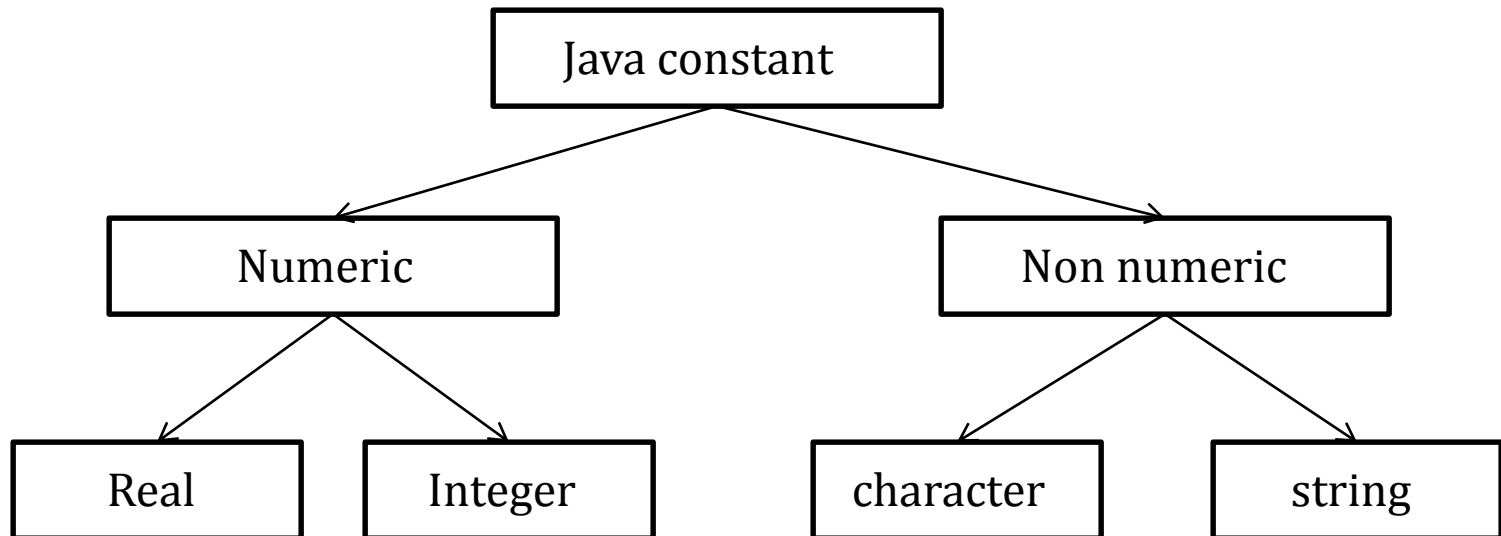
- Constant in java refers to fixed values do not change during execution of program.
- 1. Integer Constant
- 2. Real constant
- 3. Single constant
- 4. String constant



1) INTEGER CONSTANT

□ Integers :

- Decimal : 0 to 9 [ex. 123, 012 , 16789]
- Octal : [ex. 037 , o ,0435]
- Hexadecimal : [0x2 , 0x9F , 0x ,0xbcd]



2) REAL CONSTANT

- Floating point / Real point constants

Ex. 0.083 , -0.75 , 215. , .95 , -.71

syntax : mantissa e Exponent

- Ex.

● .65e4 , 12e-4 , 1.5 e+5 , 3.18E3

- Floating point contain 4 parts:

1. A whole number
2. A decimal number
3. A fractional part
4. An exponent



3) SINGLE CHARACTER CONSTANT

Ex.

'5' , 'Y' , ';' , ''

4) STRING CONSTANT

- Sequence of character enclosed between **double quotes.**
- Ex.
"hello java" , "1997","?----" , "5+3" , "x"



BACK SLASH CHARACTER SEQUENCE (ESCAPE SEQUENCE)

'\b'	Back space
'\f'	Form feed
'\n'	New line
'\r'	Carriage return
'\t'	Horizontal tab
'\''	Single quote
'\"'	Double quote



2.5 TYPE CONVERSION AND CASTING

□ Process of converting one data type to another is called type casting

□ When we need casting?

it is used when there is need to store a value of one type into a variable of another type.

□ **Syntax:**

type variable1 = (type) variable2 ;

Example:

`int m=5;`

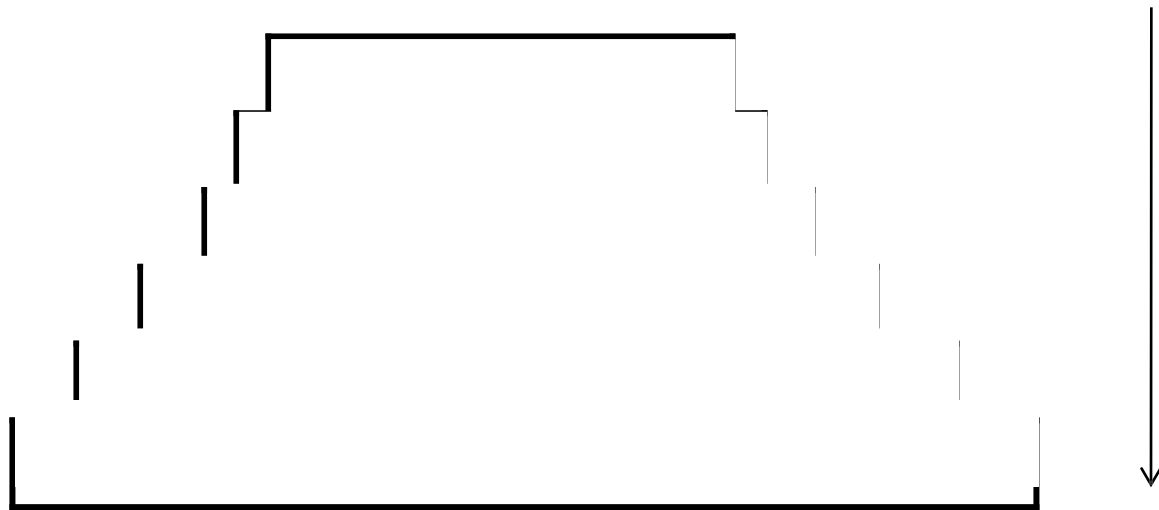
`byte n=(byte) m; // converts int m into byte type`

`count=(long) m; // converts int m into long`

type



- float and double type can be converted any except boolean.
- byte, short, int and long can be converted in any type except boolean
- Casting in smaller type may result in loss of data.
- Casting floating point into integer will loss of fractional part.



From	To
byte	short , char , int , long , float , double
short	int , long , float , double
char	int , long , float , double
int	long , float , double
long	float , double
float	double

Type casting with out loss of data



Type casting in Java

is a fundamental concept that allows developers to convert data from one data type to another.

type casting is a method or process that converts a data type into another data type in both ways manually and automatically. The automatic conversion is done by the compiler and manual conversion performed by the programmer

Rules of Typecasting

Widening Conversion (Implicit)

- No explicit notation is required.
- Conversion from a smaller data type to a larger data type is allowed.
- No risk of data loss.

Narrowing Conversion (Explicit)

- Requires explicit notation using parentheses and casting.
- Conversion from a larger data type to a smaller data type is allowed.
- Risk of data loss due to truncation.



- **Widening Casting** (automatically) - converting a smaller type to a larger type size

byte -> short -> char -> int -> long -> float -> double

- **Narrowing Casting** (manually) - converting a larger type to a smaller size type

double -> float -> long -> int -> char -> short -> byte



Widening Casting

Widening casting is done automatically when passing a smaller size type to a larger size type:

```
public class Main
{ public static void main(String[] args)
  { int myInt = 9;
    double myDouble = myInt; // Automatic casting: int to double
    System.out.println(myInt); // Outputs 9
    System.out.println(myDouble); // Outputs 9.0
  }
}
```



Narrowing Casting

Narrowing casting must be done manually by placing the type in parentheses `()` in front of the value:

```
public class Main
{
    public static void main(String[] args)
    {
        double myDouble = 9.78d;
        int myInt = (int) myDouble; // Manual casting: double to int
        System.out.println(myDouble); // Outputs 9.78
        System.out.println(myInt); // Outputs 9
    }
}
```



2.6 STANDARD DEFAULT VALUES

Type	value
byte	Zero:(byte) 0
short	Zero:(short) 0
int	Zero : 0
long	Zero: 0L or 0l
double	0.0d
float	0.0f
char	Null character
boolean	false
Reference	null



2.7 WRAPPER CLASS

- ❑ Vector class cannot handle primitive data types like int, float, long, char and double.

Use:

- ❑ Primitive data types may be converted into objects types by using wrapper classes contained in **java.lang**;
- ❑ **Wrapper classes for converting simple types**

No.	Primitive data type	Wrapper class
1	boolean	Boolean
2	char	Character
3	double	Double
4	float	Float
5	int	Integer
6	long	Long



- ❑ Wrapper classes have number of unique methods for handling primitive data type.

Converting primitive numbers to object number Using constructor methods :

1	Integer intVal= new Integer (i);	Primitive integer to Integer object
2	Float floatVal=new Float (f);	Primitive float to Float object
3	Double doubleVal=new Double (d);	Primitive double to Double object
4	Long longVal=new Long (l);	Primitive long to Long object

Wrapper Class

A Wrapper class in Java is one whose object wraps or contains primitive data types. When we create an object to a wrapper class

In other words, we can wrap a primitive value into a wrapper class object.



Need of Wrapper Classes

They convert primitive data types into objects. Objects are needed if we wish to modify the arguments passed into a method (because primitive types are passed by value).

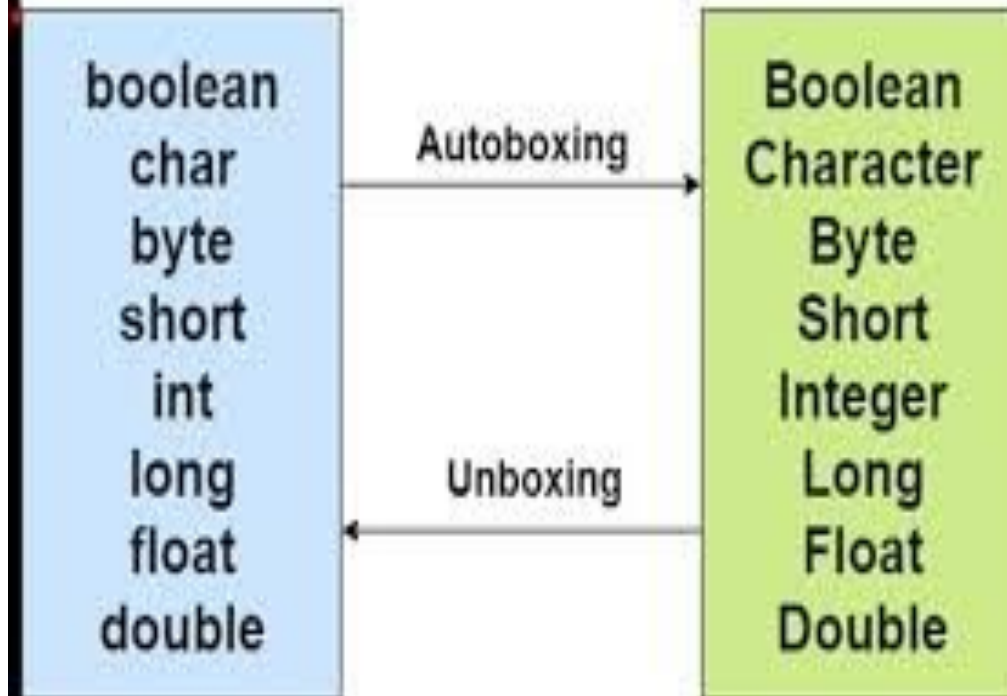
The classes in `java.util` package handle only objects

Data structures in the Collection framework, such as ArrayList and Vector, store only objects (reference types) and not primitive types.



Primitive Data Type	Wrapper Class
char	Character
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean

Wrapper Classes



Autoboxing

1. Autoboxing

The automatic conversion of primitive types to the object of their corresponding wrapper classes is known as autoboxing. For example – conversion of int to Integer, long to Long, double to Double, etc.

```
import java.util.ArrayList;
class Autoboxing {
    public static void main(String[] args)
    {
        char ch = 'a';

        // Autoboxing- primitive to Character object
        // conversion
        Character a = ch;

        ArrayList<Integer> arrayList
            = new ArrayList<Integer>();

        // Autoboxing because ArrayList stores only objects
        arrayList.add(25);

        // printing the values from object
        System.out.println(arrayList.get(0));
    }
}
```



2. Unboxing

It is just the reverse process of autoboxing. Automatically converting an object of a wrapper class to its corresponding primitive type is known as unboxing. For example – conversion of Integer to int, Long to long, Double to double, etc.

```
// Java program to demonstrate Unboxing
import java.util.ArrayList;

class Unboxing {
    public static void main(String[] args)
    {
        Character ch = 'a';

        // unboxing - Character object to primitive
        // conversion
        char a = ch;

        ArrayList<Integer> arrayList
            = new ArrayList<Integer>();
        arrayList.add(24);

        // unboxing because get method returns an Integer
        // object
        int num = arrayList.get(0);

        // printing the values from primitive data types
        System.out.println(num);
    }
}
```



```
public class WrapperExample {  
    public static void main(String[] args) {  
        // Creating Wrapper Class objects  
        Integer intObj = 100; // Autoboxing (int to Integer)  
        Double doubleObj = Double.valueOf(3.14);  
        Boolean boolObj = true; // Autoboxing (boolean to Boolean)  
  
        // Unboxing (Wrapper to Primitive)  
        int intValue = intObj;  
        double doubleValue = doubleObj;  
        boolean boolValue = boolObj;  
  
        // Printing values  
        System.out.println("Integer Object: " + intObj);  
        System.out.println("Double Object: " + doubleObj);  
        System.out.println("Boolean Object: " + boolObj);  
  
        System.out.println("Unboxed Integer: " + intValue);  
        System.out.println("Unboxed Double: " + doubleValue);  
        System.out.println("Unboxed Boolean: " + boolValue);  
  
        // Using Wrapper Classes in Collections  
        java.util.ArrayList<Integer> list = new java.util.ArrayList<>();  
        list.add(10); // Autoboxing  
        list.add(20);  
  
        System.out.println("ArrayList: " + list);  
    }  
}
```



2.8 COMMENT SYNTAX

□ Java has three types of comment lines:

- 1) **Single line comments:** start with `//` and extend to the end of the current line. This comment type is also present in C++ and in modern C

Syntax: `// This is an end-of-line comment`

- 2) **Multiline comments:** start with `/*` and end with `*/`, they may cover multiple lines. This type of comment was derived from C and C++.

Syntax : `/* This is a multi-line comment.`

□ It may occupy more than one line. `*/`



3)Documentation comments are processed by Javadoc tool to generate documentation from source files.

- it starts with `/**` and follows conventions defined by the Javadoc tool. Technically these comments are a special kind of traditional comment and they are not specifically defined in the language specification.

Syntax :

```
/**
```

```
 * This is a documentation comment.
```

```
 *
```

```
 * @author John Doe
```

```
 */
```



2.9 GARBAGE COLLECTION

- ❑ Java runtime system performs memory management.
- ❑ Java uses constructor to initialize object (give space in memory).
- ❑ Java does not have destructor as in C++.
- ❑ Java system has garbage collector and finalizer method for memory de allocation for any object.



FINALIZER METHOD

- ❑ Finalizer releases the objects special memory.
- ❑ Constructor method is used to initialize an object when it is declared, this is called initialization. Java also supports finalization, which is opposite to initialization.
- ❑ Java run time is an automatic garbage collecting system, which automatically frees up the memory resources used by objects.
- ❑ But if objects may hold other non-object resources (such as file descriptor or windows character system).
- ❑ Garbage collector can't free these resources, to free these resources we must use Finalizer methods.



FINALIZER METHOD

- ❑ **finalize()** can be added to any class.
- ❑ Java calls that method whenever it is about reclaim space for that object. **finalize()** method should explicitly define tasks to be performed.
- ❑ **Before an object gets garbage collected ,the garbage collector gives the object an opportunities to clean up itself through call to the object's finalize () method , this is called finalization.**
- ❑ When there is no more reference to object, object is finalized and is then garbage collected.
- ❑ **finalize ()** method is member of object class.



FINALIZER METHOD

- finalize () is only called just prior to garbage collector.

System . runFinalization ();

- this method force object's finalization on it finalize all objects that are waiting to be garbage collected.

Syntax:

```
protected void finalize( )  
{  
    //finalization code  
}
```

- Protected prevents access to finalize () by code defined outside its class.



GARBAGE COLLECTOR

- ❑ A Java runtime environment deletes objects periodically when it determines that they are no longer being used, this process is called ***garbage collection***.
- ❑ It is done by **garbage collector** of java.
- ❑ Garbagecollector only knows how to release memory allocated with new (means dynamically).
- ❑ An object is eligible for garbage collection when there are no more reference to that object.
- ❑ When garbage collector is ready to release the storage used for your object, it will first call finalize () and only on the next pass garbage collection pass will reclaim the objects memory.



GARBAGE COLLECTOR

- ❑ Before an object gets garbage collected ,garbage collector gives the object an opportunity to clean up itself through a call to objects finalize() method.
- ❑ Running of garbage collector:
- ❑ We can ask garbage collector to run at any time by calling gc() method of System class.

System.gc();



2.10 ARRAYS OF PRIMITIVE DATA TYPES

- Java has eight basic (primitive) data types. We can create array of any primitive data type.

Syntax: datatype Arrayname[] = new datatype[size];

Where ,size is any decimal number, it defines no..of element you want to store in that array.

Example: int a[] = new int [100];
byte val [] = new byte [10] ;
float weight[] = new
float[10];



ARRAY

- Array is a fixed size sequential collection of elements having same data type.
- We can declare array without giving its size.
(we do not have to provide array size at the time of declaration of array.)

Declaration of array:

type arrayname[] ;

ex.: int a[] ;

or

type[] arrayname;

ex.: int[] a;



Creation of array:

type Arrayname=new type [size] ;

Ex: int a=new int [10] ;

Initialization of array:

- 1) Arrayname[subscript]=value;
- 2) Type arrayname[]={ list of values};

Ex 1:

int a [0] =16;

Ex 2:

int a [] ={1,2,3,4,5,6};



ARRAY LENGTH

- In java all arrays store the allocated size in variable named **length**.
- We can access length of array using **length** variable. **int**

```
size= a.length;
```



2.11 TYPES OF ARRAYS

- Array is a fixed size sequential collection of elements having same data type.
- We can declare array without giving its size.(we do not have to provide array size at the time of declaration of array.)
- **Three types of array :**
 - 1) one dimensional
 - 2) two dimensional
 - 3) variable sized array(multi dimensional)



1) ONE DIMENSIONAL ARRAY

Declaration of array:

type arrayname[] ;

Example: int a[];

OR

type[] arrayname;

Example: int[] a;

Creation of array:

Arrayname=new type [size] ;

Example: a=new int [10] ;



ONE DIMENSIONAL ARRAY

Initialization of array:

Compile time :

- 1) Arrayname[subscript]=value;
- 2) Type arrayname[]={ list of values};

Ex:int a [] ={1,2,3,4,5,6};
int a[0]=1;
int a[1]=2; and so on

Run time :

Using command line argument , readLine() method
,using Scanner class's methods



Declaration

int[] arr; // Declaration of a 1D array of integers

Initialization

int[] arr = {10, 20, 30, 40, 50}; // Array initialized with values

Using new keyword (with predefined size)

int[] arr = new int[5]; // Array with 5 elements, default value 0

Accessing Array Elements:

```
int[] arr = {10, 20, 30, 40, 50};  
System.out.println(arr[0]); // Output: 10  
System.out.println(arr[3]); // Output: 40
```



```
public class ArrayExample {  
    public static void main(String[] args) {  
        // Declaration and initialization of a 1D array  
        int[] arr = {10, 20, 30, 40, 50};  
  
        // Accessing elements using indices  
        System.out.println("First element: " + arr[0]); // Output: 10  
        System.out.println("Second element: " + arr[1]); // Output: 20  
  
        // Modifying an element  
        arr[2] = 35; // Changing the third element from 30 to 35  
  
        // Printing the updated array  
        System.out.println("Updated third element: " + arr[2]); // Output: 35  
  
        // Iterating over the array using a for loop  
        System.out.println("Array elements:");  
        for (int i = 0; i < arr.length; i++) {  
            System.out.println(arr[i]);  
        }  
    }  
}
```



2) TWO DIMENSIONAL ARRAY

□ `int a[ ] [ ] = new int [3] [4] ;`

● 3X4 array

□ `int a[2][3]={0,0,0,1,1,1};`

□ `int a[2][3]={ {0,0,0} , {1,1,1} };`

0	0	0
1	1	1



3) VARIABLE SIZED ARRAYS

- Java treats multidimensional array as “arrays of arrays”
- It is possible to declare a two dimensional .
- `int a[ ][ ] = new int [3][ ]`

`X[0] = new int[2];`

`X[1] = new int[4];`

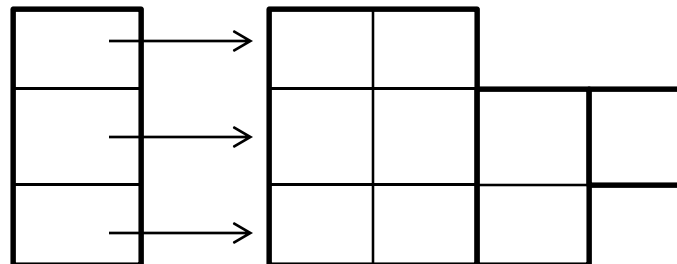
`X[2] = new int[3];`

This statements create a two dimensional array as having different length for each row as follows:

`X[0]`

`X[1]`

`X[2]`



Declaration and Initialization of a 2D Array

// Declaration of a 2D array

```
int[][] array = new int[3][4]; // 3 rows and 4 columns
```

// Initialization with values

```
int[][] array2 = {  
    {1, 2, 3, 4},  
    {5, 6, 7, 8},  
    {9, 10, 11, 12}  
};
```



Accessing Elements of a 2D Array

```
public class TwoDArrayExample {  
    public static void main(String[] args) {  
        // Initialization  
        int[][] array = {  
            {1, 2, 3, 4},  
            {5, 6, 7, 8},  
            {9, 10, 11, 12}  
        };  
  
        // Accessing elements (row index, column index)  
        System.out.println("Element at [0][0]: " + array[0][0]); // Output: 1  
        System.out.println("Element at [2][3]: " + array[2][3]); // Output: 12  
  
        // Modifying an element  
        array[1][2] = 99; // Modify element at row 1, column 2  
        System.out.println("Modified element at [1][2]: " + array[1][2]); // Output: 99  
    }  
}
```



Iterating through a 2D Array

```
public class TwoDArrayIteration {  
    public static void main(String[] args) {  
        int[][] array = {  
            {1, 2, 3, 4},  
            {5, 6, 7, 8},  
            {9, 10, 11, 12}  
        };  
  
        // Iterating through 2D array  
        for (int i = 0; i < array.length; i++) { // Loop through rows  
            for (int j = 0; j < array[i].length; j++) { // Loop through columns  
                System.out.print(array[i][j] + " "); // Print elements  
            }  
            System.out.println(); // Move to the next row  
        }  
    }  
}
```



2.12 STRING CONCEPT

- **String represents a sequence of characters.** The easiest way to represent a sequence of character in java is character array.

E.g. : `char charArray[ ]= new char[4];`
`charArray[ 0]= 'J' ,charArray[1 ]='a' ,`
`charArray[ 2]='v' ,charArray[ 3]='a'`

- character array have the ability to query their length ,but they are not good enough to support the range of operations we want to perform on strings.(like to copy one character array into another array may require a lot of efforts)
- Java can handle these problem more efficiently using **string**.



STRING CONSTRUCTOR

- ❑ In java strings are class objects which is implemented using two classes **String** and **StringBuffer**.
- ❑ Java string is an instantiated object of the **String** class.
- ❑ All string literals in Java programs, are implemented as instances of String class. Strings in Java are 16-bit Unicode.
- ❑ In JDK 1.5+ you can use StringBuilder, which works exactly like StringBuffer, but it is faster and not thread-safe



CHARACTERISTICS / BENEFITS OF JAVA'S STRING:

- More reliable and predictable .
- No problem of bound checking same as in C.
- Java string is not character array and is not NULL terminated.



valueOf() Method

The `String.valueOf()` method can be used to convert different data types into a string.

```
int num = 123;  
String str = String.valueOf(num);  
System.out.println(str); // Output: "123"
```

- ❑ **static String .valueOf (boolean b)**
- ❑ **static String .valueOf (char c)**
- ❑ **static String .valueOf (int i)**
- ❑ **static String .valueOf (long l)**
- ❑ **static String .valueOf (float f)**
- ❑ **static String .valueOf (double d)**

Conversion to String

```
StringBuffer sb = new StringBuffer("Hello");  
String str = sb.toString();  
System.out.println(str);
```



Concatenation using append()

append() adds text to the end of the StringBuffer.

```
StringBuffer sb = new StringBuffer("Hello");  
sb.append(" World");  
System.out.println(sb); // Output: Hello World
```

```
String result = String.join(" ", "Hello", "World", "Java");  
System.out.println(result);  
// Output: Hello World Java
```

Convert to Lowercase & Uppercase

StringBuffer does not have direct methods for toLowerCase() and toUpperCase(), so we convert it to a String first.

```
StringBuffer sb = new StringBuffer("Java");  
String lower = sb.toString().toLowerCase();  
String upper = sb.toString().toUpperCase();  
System.out.println(lower); // Output: java  
System.out.println(upper); // Output: JAVA
```



5. String Comparison

StringBuffer does not have a `compareTo()` or `equals()` method, so conversion to `String` is required.

```
StringBuffer sb1 = new StringBuffer("Hello"); StringBuffer sb2 = new  
StringBuffer("Hello");  
System.out.println(sb1.toString().equals(sb2.toString()));  
// Output: true
```

```
compareTo()  
String str1 = sb1.toString();  
String str2 = sb2.toString(); System.out.println(str1.compareTo(str2)); //  
Output: 0 (as both are equal)
```



Ignoring Case with equalsIgnoreCase()

```
String s1 = "Hello";  
String s2 = "hello"; System.out.println(s1.equalsIgnoreCase(s2)); // Output:  
true
```

6. Check Start and End of String

•startsWith()

```
String str = "Java Programming";  
System.out.println(str.startsWith("Java"));  
// Output: true
```

•endsWith()

```
java  
System.out.println(str.endsWith("Programming"));  
// Output: true
```



7. Character at a Specific Index (charAt())

```
StringBuffer sb = new StringBuffer("Java");  
System.out.println(sb.charAt(1));  
// Output: a
```

8. Insert Characters at a Specific Index (insert())

```
StringBuffer sb = new StringBuffer("Java");  
sb.insert(4, "Script");  
System.out.println(sb);  
// Output: JavaScript
```

9. Replace Characters (replace())

```
StringBuffer sb = new StringBuffer("Hello World");  
sb.replace(6, 11, "Java");  
System.out.println(sb); // Output: Hello Java
```

10. Delete Characters (delete())

```
StringBuffer sb = new StringBuffer("Hello World"); sb.delete(5, 11);  
System.out.println(sb); // Output: Hello
```



11. Reverse a String (reverse())

```
StringBuffer sb = new StringBuffer("Java");  
sb.reverse();  
System.out.println(sb); // Output: avaJ
```

12. Check Capacity (capacity())

The initial capacity of StringBuffer is 16 characters, which expands dynamically.

```
StringBuffer sb = new StringBuffer();  
System.out.println(sb.capacity()); // Output: 16  
sb.append("Java is fun!");  
System.out.println(sb.capacity()); // Output: 16 (if within limit)
```

13. Ensure Capacity (ensureCapacity())

```
StringBuffer sb = new StringBuffer();  
sb.ensureCapacity(50);  
System.out.println(sb.capacity()); // Output: 50
```



STRING CONCATENATION

concat() method :

Concatenates str1 and str2.

E.g. : **String** **str1**="Hello " ;

String **str2**="world" ;

s3=str1.concat(str2); //output will be
"HelloWorld"



DIFFERENCE :STRING VS. STRINGBUFFER

String class

- ❑ It creates fixed length string.
- ❑ It can not be changed.
- ❑ We can not insert characters and substrings in the middle of string.

StringBuffer class

- ❑ It creates strings of flexible (variable) length.
 - ❑ It can modified in terms of length and content.
 - ❑ We can **insert characters and substrings in the middle of a string ,or append another string to the end.**
- 

Difference between string and string buffer

```
public class StringVsStringBuffer {  
    public static void main(String[] args) {  
  
        // Using String (Immutable)  
        String str = "Hello";  
        str = str + " World"; // Concatenating to String (creates new object)  
        System.out.println("String Result: " + str); // Output: Hello World  
  
        // Using StringBuffer (Mutable)  
        StringBuffer sb = new StringBuffer("Hello");  
        sb.append(" World"); // Appending to StringBuffer (modifies existing  
object)  
        System.out.println("StringBuffer Result: " + sb); // Output: Hello World  
    }  
}
```



1.String (Immutable)

- When you concatenate two strings using the + operator, a new String object is created. The original str object remains unchanged, and a new object with the updated content is assigned to str.
- In the example, "Hello" and " World" are concatenated, and a new string is created.

2.StringBuffer (Mutable)

- The append() method modifies the existing StringBuffer object itself without creating a new one. This is more memory-efficient, especially when performing multiple string operations.
- In the example, "Hello" and " World" are appended directly to the existing StringBuffer object.



2.13 OPERATORS

- Arithmetic operators
- Relational operators
- Logical operators
- Assignment operators
- Increment / decrement operators
- conditional operators
- Bitwise operators



ARITHMETIC OPERATORS

□ $+$, $-$, $*$, $/$, $\%$

□ **For Example** $a=14$, $b=9$

● $a - b = 10$

● $a + b = 18$

● $a * b = 56$

● $a / b = 3$

● $a \% b = 2$

● $-14 \% 3 = -2$

$(a \% b = (a - (a / b) * b))$

● $14 \% 3 = 2$

● $15 / 10 = 1$

● $15 / 10.0 = 1.5$



```
import java.util.Scanner;
```

```
public class ArithmeticOperators {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
  
        System.out.print("Enter the first number: ");  
        double num1 = sc.nextDouble();  
  
        System.out.print("Enter the second number: ");  
        double num2 = sc.nextDouble();  
  
        double sum = num1 + num2;  
        double difference = num1 - num2;  
        double product = num1 * num2;  
        double quotient = num1 / num2;  
  
        System.out.println("The sum of the two numbers is: " + sum);  
        System.out.println("The difference of the two numbers is: " + difference);  
        System.out.println("The product of the two numbers is: " + product);  
        System.out.println("The quotient of the two numbers is: " + quotient);  
    }  
}
```



RELATIONAL OPERATOR

- $<$, $<=$, $>=$, $>$, $==$, $!=$
- returns true or false
- Arithmetic operator has higher priority then relational operator.
- **Example:**
 - if $a > b$
 - if $a < b$
 - if $a == b$ etc



```
import java.util.Scanner;
```

```
public class RelationalOperators {  
    public static void main(String[] args) {
```

```
        Scanner scan = new Scanner(System.in);
```

```
        //System.out.println("Enter first number: ");  
        // int num1 = scan.nextInt();
```

```
        // System.out.println("Enter second number: ");  
        // int num2 = scan.nextInt();
```

```
        int num1 = 1;  
        int num2 = 2;
```

```
        System.out.println("num1 > num2 is " + (num1 > num2));  
        System.out.println("num1 < num2 is " + (num1 < num2));  
        System.out.println("num1 >= num2 is " + (num1 >= num2));  
        System.out.println("num1 <= num2 is " + (num1 <= num2));  
        System.out.println("num1 == num2 is " + (num1 == num2));  
        System.out.println("num1 != num2 is " + (num1 != num2));
```

```
    }  
}
```



LOGICAL OPERATOR

- `&&` : logical AND
- `||` : logical Or
- `!` : logical not
- **Example:**

`if (a>b)&&(a>c)`

`if (a>b)|| (a>c)`



```
public class LogicalOperators {  
    public static void main(String[] args) {  
        boolean a = true;  
        boolean b = false;  
  
        System.out.println("a: " + a);  
        System.out.println("b: " + b);  
        System.out.println("a && b: " + (a && b));  
        System.out.println("a || b: " + (a || b));  
        System.out.println("!a: " + !a);  
        System.out.println("!b: " + !b);  
    }  
}
```



INCREMENT / DECREMENT

- Pre increment : $++m$
- Post increment : $m++$
- Pre decrement : $--m$
- Post decrement: $m--$

- **Example:**

- $B=++a;$

- $C=a++;$

- $D=a--;$



public class AssignmentOperatorsExample {
 public static void main(String[] args) {
 Assignment operator
 int x = 10;

```
        int y = x;  
        System.out.println("y = " + y);  
        x += 5;  
        System.out.println("x += 5: " + x);  
        x -= 3;  
        System.out.println("x -= 3: " + x);  
        x *= 2;  
        System.out.println("x *= 2: " + x);  
        x /= 4;  
        System.out.println("x /= 4: " + x);  
        x %= 3;  
        System.out.println("x %= 3: " + x);  
        x <<= 2;  
        System.out.println("x <<= 2: " + x);  
        x >>= 1;  
        System.out.println("x >>= 1: " + x);  
        x &= 5;  
        System.out.println("x &= 5: " + x);  
  
        x |= 2;  
        System.out.println("x |= 2: " + x);  
        x ^= 1;  
        System.out.println("x ^= 1: " + x);  
    }  
}
```



$y = 10$

$x += 5: 15$

$x -= 3: 12$

$x *= 2: 24$

$x /= 4: 6$

$x \% = 3: 0$

$x << = 2: 0$

$x >> = 1: 0$

$x \& = 5: 0$

$x |= 2: 2$

$x ^ = 1: 3$



CONDITIONAL OPERATOR

- It is also known as Ternary Operator.

- **Syntax:**

(Condition)?True Statement:False Statement.

- **Example:**

Max=(a>b)?a:b;



```
public class TernaryOperatorExample
{
public static void main(String args[])
{
int x, y;
x = 20;
y = (x == 1) ? 61: 90;
System.out.println("Value of y is: " + y);
y = (x == 20) ? 61: 90;
System.out.println("Value of y is: " + y);
}
}
```



BITWISE OPERATOR

- `&` : bitwise AND
- `!` : bitwise OR
- `^`: bitwise exclusive OR
- `v`: one's complement
- `<<` : shift left
- `>>` : shift right
- `>>>`: shift right with zero fill



```
// Java program to illustrate
```

```
// bitwise operators
```

```
public class operators {
```

```
    public static void main(String[] args)
```

```
    {
```

```
        // Initial values
```

```
        int a = 5;
```

```
        int b = 7;
```

```
        // bitwise and
```

```
        // 0101 & 0111=0101 = 5
```

```
        System.out.println("a&b = " + (a & b));
```

```
        // bitwise or
```

```
        // 0101 | 0111=0111 = 7
```

```
        System.out.println("a|b = " + (a | b));
```

```
        // bitwise xor
```

```
        // 0101 ^ 0111=0010 = 2
```

```
        System.out.println("a^b = " + (a ^ b));
```

```
        // bitwise not
```

```
        // ~00000000 00000000 00000000 00000101=11111111 11111111 11111111
```

```
11111010
```

```
        // will give 2's complement (32 bit) of 5 = -6
```

```
        System.out.println("~a = " + ~a);
```

```
    }
```

```
}
```



Mathematical Function

Method	Description
<u>sin</u>	Returns the trigonometric value of the sine of an angle.
<u>cos</u>	Returns the trigonometric value of the cosine of an angle.
<u>tan</u>	Returns the trigonometric value of the tangent of an angle.
<u>exp</u>	Returns Euler's number e raised to the power of a double value
<u>log</u>	Returns the natural logarithm (base e) of a double value
<u>sqrt</u>	Returns the correct rounded positive square root of a double value
<u>cbrt</u>	Returns the cube root of a double value
<u>ceil</u>	Returns the smallest double value that is greater than or equal to the argument. It is a fixed mathematical integer
<u>floor</u>	Returns the smallest double value that is less than or equal to the argument. It is a fixed mathematical integer.



<u>pow</u>	Returns(pow(a,b)) the value of ab.
<u>round</u>	Returns the closest int to the argument, with ties rounding to positive infinity
<u>abs</u>	Return the absolute value
<u>max</u>	Returns the maximum out of the two arguments
<u>min</u>	Returns the minimum out of the two arguments



```
import java.util.Scanner;
public class MathFunctions
{
    public static void main(String[] args)
    {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a number: ");
        double number = scanner.nextDouble();
        System.out.println("Sine: " + Math.sin(number));
        System.out.println("Cosine: " + Math.cos(number));
        System.out.println("Tangent: " + Math.tan(number));
        System.out.println("Exponential (e^x): " + Math.exp(number));
        System.out.println("Natural Logarithm (ln): " + Math.log(number));
        System.out.println("Square Root: " + Math.sqrt(number));
        System.out.println("Cube Root: " + Math.cbrt(number));
        System.out.println("Ceiling: " + Math.ceil(number));
        System.out.println("Floor: " + Math.floor(number));
        scanner.close();
    }
}
```



2.14 JAVA STATEMENTS

- Iteration statement:

Three types

- While , do , For

- Jump statement:

Four types

- Break , Continue , Return , throw

- Synchronization statement: used for handling issues with multithreading

- Guarding statement: used for safe handling of code that may cause exceptions

- Try, catch , finally



JAVA CONTROL STATEMENTS

Java control statements are of three type:

- 1) Selection Statement (if, if...else, switch)
- 2) Loops while, do-while, for)
- 3) Jump statements (break, continue, return & exit)



SELECTION STATEMENT:

1) simple if statement :

```
if ( test expression)
{
    //statement block;
}
```



```
public class Student {  
public static void main(String[] args) {  
int x = 10;  
int y = 12;  
if(x+y > 20) {  
    System.out.println("x + y is greater than 20");  
}
```

```
int x = 20;  
int y = 18;  
    if (x > y)  
{  
    System.out.println("x is greater than y");  
}
```



SELECTION STATEMENT:

2) if...else statement :

if (test expression)

{

 //true block statement block;

}

else

{

 // false block statement;

}



```
int time = 20;
if (time < 18)
{
    System.out.println("Good
day.");
}
Else
{
    System.out.println("Good
evening.");
}
```

```
public class Student {
    public static void main(String[] args)
    {
        int x = 10;
        int y = 12;
        if(x+y < 10)
        {
            System.out.println("x + y is less than
0");
        }
        else
        {
            System.out.println("x + y is greater tha
20");
        }
    }
}
```



SELECTION STATEMENT:

3) nesting of if.....else statement:

```
if ( test condition-1)
{
    if(condition-2)
    {
        statement-1;
    }
    else
    {
        Statement-2;
    }
}
else
{
    Statement-3;
}
```



```
public class A{
    public static void main(String[] args)
    {

        int n=23;

        //if else condition to check whether the number is even or odd
        if (n % 2 == 0){

            //the number is even
            System.out.print("Even ");

            //nested if else condition to check if n is divisible by 6 or not
            if (n % 6 == 0) {
                //the number is divisible by 6
                System.out.println("and divisible by 6");
            } else {
                //the number is not divisible by 6
                System.out.println("and not divisible by 6");
            }
        }

        else {

            //the number is odd
            System.out.println("Odd ");

        }

    }
}
```



SELECTION STATEMENT:

4) else....if ladder

```
if ( condition-1)
{
    statement-1;}
else if ( condition 2)
{
    statement-2;
}
else if ( condition 3)
{
    statement-3;}
else
    default: default    statement;
```



```
import java.util.Scanner;
class Test
{ // main() method
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter a percentage");
float per = sc.nextFloat();
if(per>=70)
{
System.out.println("Distinction");
}
else if(per>=60)
{
System.out.println("First Class");
}
else if(per>=50)
{ System.out.println("Second Class");
}
else if(per>=40)
{ System.out.println("Pass");
}
else
{ System.out.println("Failed");
}
}
}
```



SELECTION STATEMENT:

5) **switch statement :**

switch(integer or character value or expression)

{

case value-1:

 block-1

 break;

case value-2:

 block-2

 break;

.....

default :

 default block;

 break; }



```
int day = 4;

switch (day)
{
case 1:
    System.out.println("Monday");
    break;
case 2:
    System.out.println("Tuesday");
    break;
case 3:
    System.out.println("Wednesday");
    break;
case 4: System.out.println("Thursday");
    break;
case 5: System.out.println("Friday");
    break;
case 6: System.out.println("Saturday");
    break;
case 7: System.out.println("Sunday");
    break;
}
```



ITERATION STATEMENT:

1) **while statement:**

```
while(test condition)
{
    Body of the loop
}
```

2) **do..while statement:**

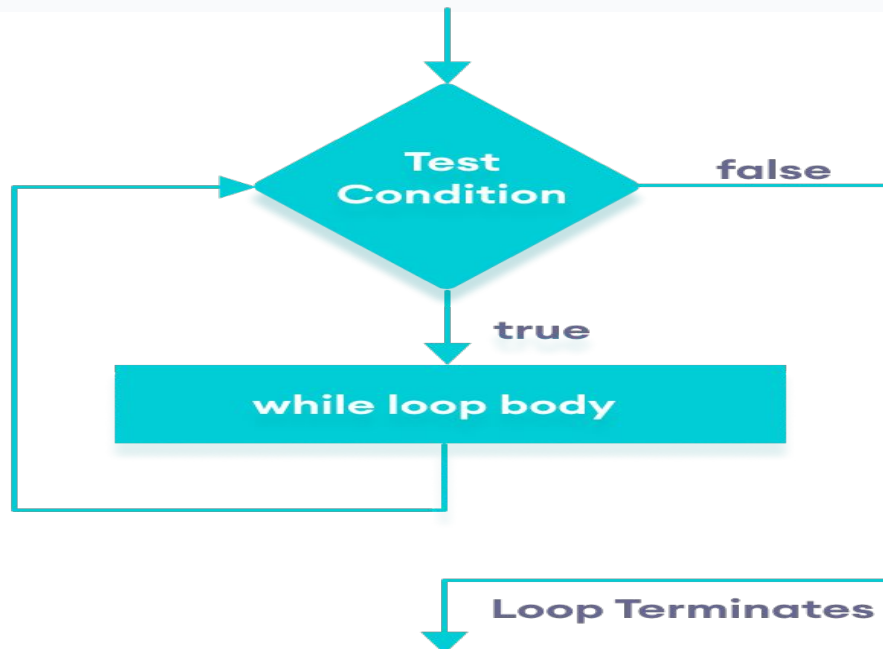
```
do
{
    Body of the loop
}while(test condition);
```



Java while loop is used to run a specific code until a certain condition is met. The syntax of the while loop is:

```
while (testExpression) { // body of loop }
```

1. A while loop evaluates the **testExpression** inside the parenthesis ().
2. If the **testExpression** evaluates to true, the code inside the while loop is executed.
3. The **testExpression** is evaluated again.
4. This process continues until the **testExpression** is false.
5. When the **testExpression** evaluates to false, the loop stops.



// Java program to find the sum of positive numbers

import java.util.Scanner;

class Main {

public static void main(String[] args) {

int sum = 0;

// create an object of Scanner class

Scanner input = new Scanner(System.in);

// take integer input from the user

System.out.println("Enter a number");

int number = input.nextInt();

// while loop continues

// until entered number is positive

while (number >= 0) {

// add only positive numbers

sum += number;

System.out.println("Enter a number");

number = input.nextInt();

}

System.out.println("Sum = " + sum);

input.close();

}

}



The example below uses a **do/while** loop. The loop will always be executed at least once, even if the condition is false, because the code block is executed before the condition is tested:

```
import java.util.Scanner;
```

```
class Main {  
    public static void main(String[] args) {
```

```
        int sum = 0;  
        int number;
```

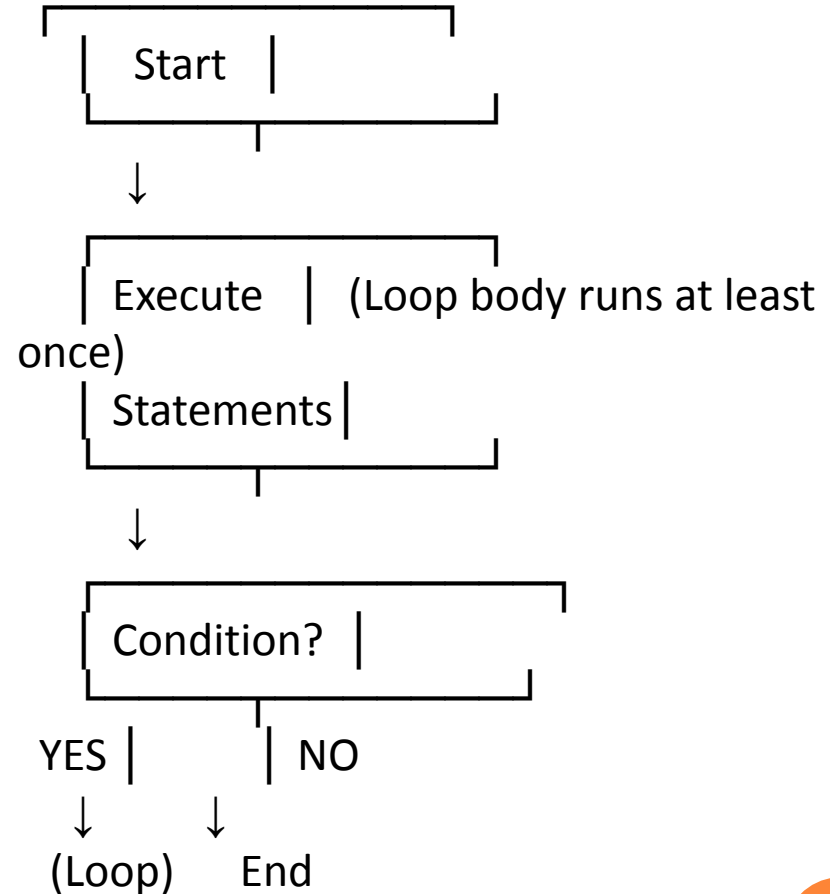
```
        // create an object of Scanner class  
        Scanner input = new Scanner(System.in);
```

```
        // do-while loop executes at least once  
        do {  
            // take integer input from the user  
            System.out.println("Enter a number");  
            number = input.nextInt();
```

```
            // add only positive numbers  
            if (number >= 0) {  
                sum += number;  
            }
```

```
        } while (number >= 0);
```

```
        System.out.println("Sum = " + sum);  
        input.close();  
    }  
}
```



ITERATION STATEMENT:

3) for loop:

```
for ( initialization ; test expression ; increment or  
decrement )
```

```
{
```

Body of the loop

```
}
```



JUMP STATEMENT

- 1) **break:** It is used to exit from a loop or block
- 2) **continue :**it is used to skip some part of code and causes loop to be continued with the next iteration.
- 3) **return :**it is used to return values to calling function



```
import java.util.Scanner;
class Main {
    public static void main(String[] args)
    {
        int sum = 0;
        Scanner input = new Scanner(System.in);
        while (true) {
            System.out.println("Enter a number");
            int number = input.nextInt();

            // If a negative number is entered, break the loop
            if (number < 0) {
                System.out.println("Negative number entered. Exiting loop...");
                break; // Terminates the loop
            }

            // Skip zero, do not add to sum
            if (number == 0) {
                System.out.println("Zero entered, skipping...");
                continue; // Skips the rest of the loop iteration
            }
            sum += number; // Add positive numbers
        }
        System.out.println("Sum = " + sum);
        input.close();
    }
}
```

